

Ch 1: C Basics, Pointers & Functions

Types & format specifiers

type	B	printf	scanf
char	1	%c	%c
int	4	%d	%d
float	4	%f	%f
double	8	%f	%lf
pointer	4 (exam) / 8 %p		

%x hex, %e sci, %s str, %zu size_t, %% = %. sizeof(char) = 1 always. int max $2^{31}-1 = 2147483647$. "[%5d]", 42 → [42]; %-5d left-justify; %8.3f width 8, 3 d.p.; %f default 6 d.p.
Trap: scanf needs addresses (&x); %f into a double is wrong.

Precedence (high → low)

() [] . -> x++ x--	L→R
* & ! ~ ++x --x (type) sizeof (unary)	R→L
* / % then + - then << >>	L→R
< <= > >= then == !=	L→R
& then ^ then then && then	L→R
?: then = += -= ...	R→L
5>3>1 = 1>1 = 0. a b&& c = a (b&&c). x&1==0 = x&(1==0).	

Division, %, casts

int/int truncates **toward 0**. Either operand real ⇒ real. % sign follows the dividend.

float p=10/4	2.0	-10/4	-2
int r=-10/4.0	-2	-10%4	-2
(float)a/4 (a=6)	1.5	10%-4	2
(float)(a/4)	1.0	1/2+1.5	1.5

++ / -- and short-circuit

i++ uses old then bumps; ++i bumps then uses.
i=0; r=(i++ || i++) → r=1, i=2. i=0; r=(i && j++) → j unchanged.

expr	means	effect (p→a[0]=10)
*p++	*(p++)	value 10, pointer moves
(*p)++		value bumped, p stays
++p	*(++p)	p moves, then read
++*p	++(*p)	value bumped, new value

Past loops: for(i=0; i<10; i++) printf("%d ", ++i); → 1 3 5 7 9.
i=5; do printf("%d ", i--); while(--i && i>0); → 5 3 1.
0 = false, non-zero = true. switch falls through until break.

Trap: if (a = 5) assigns (always true). if() / if(;) are syntax errors; if(i?0:1) is valid.

Pointers

&x = address of x; *p = what p points to. int *p=&a; ⇒ *p≡a.
p+k moves $k \times \text{sizeof}(*p)$ bytes (int/float +4, char +1, double +8).

int *p, q; → q is a plain int. int *n; *n=123; → seg fault (uninitialised).

Tracing: one column per variable; for pointers write *what they point to*; on *p = ... update the pointee's column.

Pointer-to-pointer (int A[4]={1,2,3,4}, *p[2], **q);
AY24/25: p[0]=&A[1]; p[1]=&A[3]; q=p; *q++=&A[2]; **q=100; → 1 2 3 100.
AY25/26: p[0]=&A[0]; p[1]=&A[2]; q=p; *q=&A[1]; **q++=100; → 1 100 3 4; after: q = p+4 B, *q = &A[2] (A+8).

Rule: *q++ derefs the **old** q, then q moves one element.

Functions, scope, static

Pass-by-value: the callee gets copies; swap(int a, int b) does nothing. Pass &a and write through *p to change the caller.

No prototype ⇒ implicit int return ⇒ “conflicting types” at a double definition.

Arrays are passed as a pointer: the callee can modify them, and sizeof(arr) inside is the pointer size.

Innermost declaration wins (shadowing). A function sees globals, not the caller's locals.

static local: initialised once, keeps its value across calls/iterations.

AY24/25 Q9-10: global x=1; main int x=2; for(int x=0; x<4; x++){int x=3; printf("%d ", x++);} then printf("%d", x) → 3 3 3 3 2. With static int x=3; → 3 4 5 6 2.

Bitwise (a=5=00000101, b=22=00010110)

a|b 23 a&b 4 | a^b 19 ~a 250
b<<2 88 b>>3 2 | <<k = $\times 2^k$, >>k = $\div 2^k$
Trap: &| are bitwise; &&|| are logical and short-circuit.

Lab facts

char *s="...": read-only, s[0]='R' → seg fault, s++ OK.
char s[]="...": writable, s++ → compile error.
gcc: -Wall, -lm (math), -g (gdb), -DNAME defines a macro.

- Exam patterns (past midterms)**
1. Loop with ++/-- output (trace i per iteration; do-while runs once first).
 2. Shadowing / static: which x each printf sees.
 3. Pointer trace table; **q++ with int *p[2].
 4. Valid syntax MCQ (if(i?0:1) ok; {1,,3,4} not).
 5. Pass-by-value vs pointer; int division and cast placement.

Ch 2: Arrays, Strings, Structs

Array initialisers

int b[] = {1,2,3};	OK, size 3
int c[5] = {17,3,10};	OK, rest = 0
int e[2] = {1,2,3};	warning (excess)
int f[5]; f = {...};	error (only at declaration)
int A[4] = [1,2,3,4];	error
int A[4] = {1, ,3,4};	error
char s[3] = "abc";	legal (no room for \0)
char s[3] = "abcd";	warning

Arrays = pointers

a[i] ≡ *(a+i); &a[i] ≡ a+i = base + $i \times \text{sizeof}(\text{elem})$
Array name is a **constant** pointer: a++, dest = src are errors.

int b[5]={10,20,30,40,50}, *q=b+1; q[2]=40, *q+1=21, *(q+1)=30.
2D int m[R][C]: &m[i][j] = m + (iC + j) × 4 (row-major).
Trap: a == b compares addresses, not contents.

sizeof

```
int x[10]; sizeof(x)      40
sizeof(x)/sizeof(x[0])  10 elements
char s[]="apple"; sizeof(s) 6 (includes \0)
strlen(s)                5
char *p="apple"; sizeof(p)  4 / 8 (pointer)
sizeof(arr) inside a function  pointer size
```

Strings

String = char array ending in `'\0'`. `'A'` = 65 (1 byte); `"A"` = 2 bytes.

ASCII `'0'`=48, `'A'`=65, `'a'`=97; lower = upper +32.

<code>fgets(s,n,stdin)</code>	$\leq n-1$ chars, keeps the <code>\n</code> , adds <code>\0</code>
<code>scanf("%s",s)</code>	one word, no <code>&</code>
<code>puts(s)</code>	prints + newline
<code>strlen(s)</code>	chars before <code>\0</code>
<code>strcmp(a,b)</code>	< 0 / 0 (equal) / > 0 , by ASCII
<code>strcpy(d,s)</code>	copies incl. <code>\0</code>
<code>strncpy(d,s,n)</code>	no <code>\0</code> if $n \leq \text{strlen}(s)$
<code>strcat(d,s)</code>	append

Strip newline: `if (s[len-1]=='\n') s[len-1]='\0';`
`strcmp("abc","ab") > 0`; `strcmp("Apple","apple") < 0`.

Trap: `s1 == s2` compares addresses; `strcmp` returns 0 when *equal*. `name = "x"` on a char array is illegal (use `strcpy`). No `\0` $\Rightarrow$ `%s/strlen` run into garbage.

Structs

```
typedef struct { int id; float score; char g; }
result_t;
result_t r = {123, 93.5, 'A'}; nested: {888,
{31,12,2020}}; missing  $\rightarrow$  0.
A typedef is a type, not a variable (no memory).
```

```
p->m  $\equiv$  (*p).m    *p.m  $\equiv$  *(p.m) (wrong!)
arr[i].m  $\equiv$  (arr+i)->m
```

<code>arr2 = arr1;</code>	illegal
<code>r2 = r1; (struct)</code>	legal , member-wise copy
struct with array member	array is copied
struct with pointer member	only the address copied (still aliases)
pass struct to function	by value (caller unchanged)
pass <code>&struct</code> / array	callee can modify

Member offset = sum of sizes before it: `struct A{A* ptr; char* str; int num;}` at 0x7FFCC8B0 $\Rightarrow$ `&num =`

0x7FFCC8B8.

Past-paper traces (answers)

(a) `struct{int numer[1]; int *denom;}` passed by value, callee writes both: array copy unchanged (1), pointer writes through (`val1=10`) $\Rightarrow$ 1 10.

(b) `changeNum(ex,888)` by value $\Rightarrow$ struct unchanged.

(c) `arrA[3]` self-pointing, `*(arrA[i].ptr)=arrA[i]`: copying a struct also copies its `ptr`, so re-read pointers each iteration $\Rightarrow$ {hello,5},{string,6},{hello,5}.

C arrays $\rightarrow$ MIPS

```
int: sll $t0,$s1,2; add $t0,$s0,$t0; lw $t1,0($t0).
char: add $t1,$s4,$s0; lb $t2,0($t1) (no scaling).
Stop at \0: bne $t2,$zero,loop.
```

Exam patterns (past midterms)

- Which initialisation is valid? (arrays, `char s[3]="abc"`) (AY24/25, AY25/26 Q7)
- `*q++` / `**q++` traces with addresses (base +4*k*).
- Struct by value vs pointer: embedded arrays copy, pointers alias.
- Array of structs + struct assignment; member address from offsets.
- C string loop $\leftrightarrow$ MIPS (1b/sb, ASCII).

Ch 3: Number Representation

Bases

$(a_n \dots a_0.f_1 \dots f_m)_R = \sum a_i R^i + \sum f_j R^{-j}$. M values need $\lceil \log_2 M \rceil$ bits.

Whole part: divide by R; first remainder = LSB.

Fraction: multiply by R; first carry = MSB.

43 $\rightarrow$ (101011)₂; 0.3125 $\rightarrow$ (.0101)₂.

2/8/16: group 3 or 4 bits outward from the point; pad with 0s at both ends.

Trap: fraction groups pad on the *right*:
.1011 1 $\rightarrow$.1011 1000 = .B8.

Trap: k digits asked $\Rightarrow$ compute $k+1$ and round.
0 0000 1 0001 2 0010 3 0011 4 0100 5 0101 6 0110 7 0111
8 1000 9 1001 A 1010 B 1011 C 1100 D 1101 E 1110 F 1111
 $2^{-1} \dots^{-4}$: .5 .25 .125 .0625; $2^{10} = 1024$, $2^{16} = 65536$,
 $2^{24} = 16777216$.

Unknown base: every digit $<$ base; convert each side to decimal and test the options.

$(135)_x + (25)_x = (161)_x \Rightarrow x = 9$.

ASCII: `'0'`=0x30, `'A'`=0x41, `'a'`=0x61, space 0x20; 7 bits + parity.

Signed integers (n bits)

Scheme	Range	Negate / zeros
Sign-mag	$\pm(2^{n-1}-1)$	flip MSB; ± 0
1s	$\pm(2^{n-1}-1)$	invert; $-x = 2^{n-1}-x$; ± 0
2s	$-2^{n-1} \dots 2^{n-1}-1$	invert +1; $-x = 2^n - x$
Excess- K	$-K \dots 2^n-1-K$	value = code $-K$
Unsigned	$0 \dots 2^n-1$	—

MSB weight: 2s -2^{n-1} (1011 = -5); 1s $-(2^{n-1}-1)$ (1011 = -4); SM 1011 = -3 .

8-bit -14 : SM 10001110, 1s 11110001, 2s 11110010.

Excess- 2^{n-1} = 2s with MSB flipped (MSB 1 = non-negative).

General: $(r-1)$'s comp = $r^n - r^{-m} - N$; r 's comp = $r^n - N$.

Fractions: 2s = invert, then +1 at the *last* bit (2^{-m}).

Sign extension: 2s/1s copy the MSB left. SM: keep the sign bit, pad the magnitude with 0s (1010 $\rightarrow$ 10000010).

Trap: largest 2s = 0 then $n-1$ ones: 137 bits $\Rightarrow$ 0x0FFF...F (34 F's). 9-bit SM max is 255.

Add / subtract

2s: add, *discard* carry out. **1s:** add, *add carry back* to the LSB.

$A - B = A + (-B)$. **Overflow** iff pos+pos $\rightarrow$ neg or neg+neg $\rightarrow$ pos (2s: carry in $\neq$ carry out of MSB).

2s 4-bit: 5 + 6 = 1011 OVF; $-7 + -3 = 1\ 0110$ OVF; $-2 + -6 = 1\ 1000 = -8$ ok.

1s 7-bit: 1011101-1001100 $\Rightarrow$ 1011101+0110011 = 1 0010000 $\rightarrow$ 0010001.

Trap: 1s with fractions: align widths first; the end-around carry goes to the LSB of the *fraction*.

C int: 2147483647+1 wraps to -2^{31} . $(X \ll 3) - X = 7X$.

Floating point

IEEE 754, fixed point, rounding, range/precision, INT8 $\rightarrow$ **FP sheet**.

Exam patterns (past midterms)

- Unknown base; n -bit range or max value (sometimes in hex, e.g. 137-bit).
- Sign extension (SM vs 2s); 1s subtraction with end-around carry.
- Decimal fraction $\rightarrow$ base r to k digits; C int wrap-around.

Ch 4: MIPS Assembly

Instructions

add/sub \$rd,\$rs,\$rt	rd = rs $\pm$ rt
addi \$rt,\$rs,C	rt = rs + SE(C)
and/or/xor/nor	bitwise; nor = ~(rs rt)
andi/ori/xori	with ZE(C)
sll/srl \$rd,\$rt,sh	shift, fill 0s
lui \$rt,C	rt = C << 16
slt/slti	1 if rs < rt (signed)
lw/sw \$rt,off(\$rs)	Mem[rs+SE(off)]
lb/sb	1 byte; lb sign-extends
beq/bne \$rs,\$rt,L	branch if = / $\neq$
j L	always

No subi, not, nori; blt/bgt/ble/bge are pseudo.

SE: addi, slti, lw/sw/lb/sb, beq/bne. **ZE**: andi, ori, xori.
ori \$t0,\$zero,0xFFFF $\rightarrow$ 0x0000FFFF; addi same $\rightarrow$ 0xFFFFFFFF.
addi \$t0,\$zero,0x9200 $\rightarrow$ 0xFFFF9200 (bit 15 set).

32-bit constant: lui \$t0,0xAAAA; ori \$t0,\$t0,0xF0F0.

Trap: addi instead of ori after lui breaks if bit 15 = 1 (upper half -1).

Idioms

move	add \$d,\$s,\$zero (or addi 0, or \$zero, sll 0)
NOT	nor \$t0,\$t0,\$zero
negate	sub \$t0,\$zero,\$t0
$\times 8 / \times 7$	sll 3 / sll 3 then sub x
$\% 2^k$	andi with 2^k-1
set / clear / toggle	ori / and with ~mask / xori
bit k	srl k; andi 1
nop	sll \$zero,\$zero,0 = 0x00000000
push	addi \$sp,\$sp,-4; sw \$x,0(\$sp)
Trap: andi \$r1,\$r2,0xFF	is NOT a move. andi also

clears bits 31..16. srl of a negative $\neq$ division.
Set bits 9, 12, 15: ori \$t0,\$zero,0x9200.

Memory

Byte-addressed; word = 4 B; consecutive words differ by 4.
Aligned $\iff$ hex ends in 0/4/8/C.
A[i] (int) at base +4*i*: sll \$t1,\$s1,2; add \$t1,\$s3,\$t1; lw \$t2,0(\$t1).
A[7]=h+A[10]: lw \$t0,40(\$s3); add \$t0,\$s2,\$t0; sw \$t0,28(\$s3).
Trap: offset must be a constant: lw \$t0,\$t1(\$s0) is illegal.

Inequalities with slt

goto L if MIPS	
$a < b$	slt \$t0,a,b; bne \$t0,\$zero,L
$a > b$	slt \$t0,b,a; bne \$t0,\$zero,L
$a \geq b$	slt \$t0,a,b; beq \$t0,\$zero,L
$a \leq b$	slt \$t0,b,a; beq \$t0,\$zero,L
$a \leq C$	slti \$t0,a,C+1; bne ...
bne after slt = branch if true; beq = branch if false. Each pseudo-branch = 2 instructions.	

Control flow

if: branch on the *negated* condition over the body.
if(i==j) f=g+h; $\rightarrow$ bne \$s3,\$s4,Exit; add ...
if-else: bne ...,Else; <then>; j Exit; Else: <else>.
Decompile: beq \$s1,\$s2,Exit; add \$s0,\$zero,\$zero $\Rightarrow$ if (i != j) f = 0;
while (top test): Loop: bne ...,Exit; <body>; j Loop.
do-while (bottom test): Loop: <body>; bne ...,Loop.

Counting executed instructions

Total = before loop + $n \times$ (per iteration, per path) + final failing test + after.
Branch *executed* $\neq$ *taken*: bottom-test **bne** runs n , taken $n-1$.
Pseudo: blt/bge/bgt/ble = 2, move = 1, li = 1 or 2.
Labels are free.

Count zeros in 40 ints: index version $325 + z$; pointer version $245 + z$; bottom-test pointer $163 + z$.

End pointer = base +4*N* bytes.
L7 Ex4: t0=0;t1=0;t2=4; Again: t1+=t0; t0++; bne t2,t0,Again $\Rightarrow 3 + 4 \times 3 = 15$ instructions, \$t1=6.
Trap: beq ...,Loop loops only while *equal*: read the branch sense.

Pseudo $\rightarrow$ real

move \$d,\$s add \$d,\$s,\$zero
bgt \$1,\$2,L slt \$at,\$2,\$1; bne \$at,\$zero,L
bge \$1,\$2,L slt \$at,\$1,\$2; beq \$at,\$zero,L
li \$r,C ori \$r,\$zero,C or lui+ori
lui without lui ori \$x,\$zero,C; sll \$x,\$x,16

Exam patterns (past midterms)

- Trace a loop: # instructions, # times a branch runs/is taken, final register.
- “Which sequence is equivalent?” $\rightarrow$ almost always a SE/ZE trap.
- Implement a pseudo-instruction without clobbering live registers.
- Fill in missing lines of C $\rightarrow$ MIPS; array index (sll+add) vs pointer (+4).
- Decompile: the branch condition is the *skip* condition.

Ch 5: Instruction Encoding & ISA

Reach (encoding itself $\rightarrow$ MIPS sheets)

Branch: imm $\in [-2^{15}, 2^{15}-1]$ words from PC+4 ($\approx \pm 128$ KB). Far target: invert the branch and hop over a j.
Jump: any word in the same 256 MB block as PC+4 (blocks start at 0x?0000000); crossing a block needs jr.
j at 0xCFFFFFFC can only reach 0xD... (top nibble of PC+4).
General: n -bit address field, 2^k -byte instructions $\Rightarrow$ reach 2^{n+k} bytes (12-bit field, 16-bit instrs: 8192 B).
Why 5-bit shamt: shifting 32-bit regs by ≥ 32 is pointless (shamt bits = log₂ reg width).

Addressing modes

MIPS: register (add), immediate (addi), base/displacement (lw/sw), PC-relative (beq), pseudo-direct (j).
Trap: L10: only 3 modes for *data* operands (register, immediate, displacement).

Register indirect (R1)	M[R1]
Indexed (R1+R2)	M[R1+R2]
Direct (1001)	M[1001]
Memory indirect @ (R3)	M[M[R3]]
Auto-inc (R2)+ / dec -(R2)	use then bump / bump then use
Scaled 100 (R2) [R3]	M[100+R2+R3×d]

Storage architectures: C = A + B

Stack	Accum.	Reg-mem	Load-store
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R1,B	Load R2,B
Add	Store C	Store R1,C	Add R3,R1,R2
Pop C			Store R3,C

Memory-memory: Add C,A,B. RISC = fixed length, load-store; CISC = variable length.

Endianness

0xDEADBEEF at 0: big = DE AD BE EF; little = EF BE AD DE.
1b 0(\$0): big 0xFFFFFDE, little 0xFFFFFEF.
Trap: only bytes swap, so parity still works across machines.

Expanding opcode

Opcode bits of a class = length – operand bits. Sort $a < b < c$.
 $N_{max} = 1 + 1 + ((2^a - 1)2^{b-a} - 1)2^{c-b}$
 $N_{min} = (2^a - 1) + (2^{b-a} - 1) + 2^{c-b}$
2 classes: $N_{max} = 1 + (2^a - 1)2^{b-a}$; $N_{min} = (2^a - 1) + 2^{b-a}$.

AY24/25: 16-bit, 3-bit regs, 3/2/1 operands $\Rightarrow a = 7$, $b = 10$, $c = 13$. Max = $1 + 1 + (127 \cdot 8 - 1) \cdot 8 = \mathbf{8122}$; min = $127 + 7 + 8 = \mathbf{142}$.

Trap: the official memo says 198 for the min, but 142 is valid and fills the space. Show working.
“At least k ”: reserve exactly k in MAX. Drawback: complex decoding.

Exam patterns (past midterms)

- Expanding opcode max and min (4–6 marks), often with “at least k ” or mixed lengths.
- Jump reach across a 256 MB boundary; how many instructions fit (2^{26} per block).
- Addressing mode of an instruction; storage-architecture code for an expression.
- MCQ: 6-bit opcodes, 32 registers, endianness and parity.

Ch 6: Datapath & Control

Control table, ALU control, muxes, wire values, latency $\rightarrow$ **datapath cards**. Below is what the cards don’t have.

Control logic

RegDst = R; ALUSrc = lw + sw; MemToReg = lw;
RegWrite = R + lw; MemRead = lw; MemWrite = sw;
Branch = beq; ALUop1 = R; ALUop0 = beq.
Funct goes only to ALU control (two-level decoding: smaller, faster main control).
X only on RegDst/MemToReg, and only when RegWrite = 0. Enables are never X.

Trap: Tut 5 uses a different column order from L12. Read the header!

Which instruction uses what

Trap: “RD2 is used by all” is FALSE (lw, addi ignore it).
sw reads 2 regs. “For some instructions RR1 is not used” is TRUE (sll, lui). Every value is still *computed*.
Separate I-Mem and D-Mem (“same memory” is FALSE); the diagram doesn’t show how instructions get loaded.
Not supported: sll/srl, j, andi/ori (always sign-extends).

Stuck / wrong signals

RegDst=0	R-type writes rt (clobbers a source)
RegDst=1	lw/addi write reg # = top 5 bits of imm
RegWrite=0	R, lw, addi lose their result
ALUSrc=1	R-type uses SE(Inst[15:0]); beq compares to imm
ALUSrc=0	lw/sw address = [rs]+[rt]
MemToReg=0	lw writes the <i>address</i>
MemWrite=1	every instruction writes memory
Branch=1	anything with result 0 branches
ALUop=00	everything adds

Method: X signals can’t hurt; RegWrite = 0 makes RegDst/MemToReg irrelevant.

Past control questions

0x10000103 at 0x491: beq \$0,\$0,0x103 $\rightarrow$ 0x491+4+0x40C = 0x8A1.
17/18 S2 Q13: make sw branch $\Rightarrow$ ALU result 0 $\Rightarrow$ sw \$x,0(\$zero); lands at PC+4.
Inverted MemToReg: breaks every RegWrite = 1 instruction (lw, add, ori).
Trap: beq \$0,\$0,0 goes to PC+4; offset -1 loops on itself.

Extensions

bne: new BranchNE; PCSrc = Branch.isZero + BranchNE.isZero.
j: {(PC+4)[31:28], Inst[25:0], 00} + extra mux + Jump signal.

andi/ori: zero-extender + new ALUop. sll: shamt path + shifter.

Exam patterns (past midterms)

- Stuck/inverted control: which instructions break.
- Modified control table: trace what an instruction now does.
- T/F facts (RD2 not used by all; separate memories).